

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems
COURSE CODE: CSA0504

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool : Debugging & Optimisation Task (Low)
Assessment Tool Weightage: 10%

QUESTIONS		
Q.No	Question	Bloom's Level
1	Identify and correct errors in a given SQL CREATE TABLE statement with incorrect syntax and missing constraints.	BL2 – Understand
2	Debug an SQL query that incorrectly uses aggregate functions without GROUP BY and fix the issue. Bloom's Level:	BL2 – Understand
3	Correct a faulty SELECT query where column names are misspelled or improperly referenced.	BL1 – Remember
4	Fix errors in an SQL query using JOIN, where the join condition is missing or incorrect.	BL3 – Apply
5	Identify and correct a subquery error where multiple values are returned instead of a single value.	BL2 – Understand
6	Debug a VIEW creation statement that contains invalid column references or syntax errors.	BL1 – Remember
7	Examine an SQL query with unnecessary conditions and optimize it by simplifying the WHERE clause.	BL3 – Apply
8	Identify violations of integrity constraints in a given table and correct them.	BL2 – Understand

9	Fix a relation that violates 1NF or 2NF by removing repeating groups or partial dependencies.	BL3 – Apply
10	Identify redundant attributes in a relation and apply basic normalization (up to 3NF) to improve design.	BL3 – Apply

Code of Conduct:

I R.Pranathi(192525076) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

RUBRICS FOR CALCULATION:

Criteria	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation
	documentation			

1. Identify and correct errors in a given SQL CREATE TABLE statement with incorrect syntax and missing constraints.

ANS:

Incorrect SQL Statement

```
CREATE TABLE Student
(
  StudentID INT,
  StudentName VARCHAR(50),
  Email VARCHAR(100),
  Age INT,
  DepartmentID INT
);
```

Errors Identified

1. No PRIMARY KEY defined for StudentID.
2. StudentName should not be NULL.
3. Email should be UNIQUE to avoid duplicate email addresses.
4. Age has no validation constraint.
5. DepartmentID should be a FOREIGN KEY referencing the Department table.

Corrected SQL Statement

Create Department Table

```
CREATE TABLE Department
(
  DepartmentID INT PRIMARY KEY,
  DepartmentName VARCHAR(50) NOT NULL
);
```

Create Student Table

```
CREATE TABLE Student
(
  StudentID INT PRIMARY KEY,
  StudentName VARCHAR(50) NOT NULL,
  Email VARCHAR(100) UNIQUE,
  Age INT CHECK (Age >= 18),
  DepartmentID INT,
  FOREIGN KEY (DepartmentID)
  REFERENCES Department(DepartmentID)
);
```

Explanation of Corrections

Error	Correction
Missing Primary Key	Added PRIMARY KEY(StudentID) to uniquely identify each student.
Missing NOT NULL	Added NOT NULL for StudentName so it cannot be empty.
Missing UNIQUE	Added UNIQUE for Email to prevent duplicate email addresses.
Missing CHECK	Added CHECK (Age >= 18) to allow only valid age values.
Missing FOREIGN KEY	Added FOREIGN KEY (DepartmentID) to maintain referential integrity with the Department table.

Conclusion

The corrected CREATE TABLE statement enforces entity integrity, domain integrity, and referential integrity by using PRIMARY KEY, NOT NULL, UNIQUE, CHECK, and FOREIGN KEY constraints. This prevents invalid, duplicate, and inconsistent data from being stored in the database.

2. Debug an SQL query that incorrectly uses aggregate functions without GROUP BY and fix the issue.

ANS:

Introduction

Aggregate functions such as COUNT(), SUM(), AVG(), MAX(), and MIN() perform calculations on multiple rows and return a single result. When aggregate functions are used together with non-aggregate columns, the GROUP BY clause is mandatory. Without GROUP BY, SQL cannot determine how to group the rows, resulting in an error.

Create Sample Table

```
CREATE TABLE Student
(
    StudentID INT PRIMARY KEY,
    StudentName VARCHAR(50),
    DepartmentID INT,
    Marks INT
);
```

Insert Sample Data

```
INSERT INTO Student VALUES
(101,'Rahul',1,85),
(102,'Priya',1,90),
(103,'Arjun',2,75),
(104,'Sneha',2,80),
(105,'Kiran',3,88);
```

Incorrect SQL Query

```
SELECT DepartmentID,
COUNT(StudentID)
FROM Student;
```

Error Analysis

The above query is incorrect because:

- COUNT(StudentID) is an aggregate function.
- DepartmentID is a normal (non-aggregate) column.
- SQL requires all non-aggregate columns in the SELECT statement to appear in the GROUP BY clause.
- Since DepartmentID is not grouped, SQL generates an error.

Typical MySQL error:

ERROR 1140 (42000):

In aggregated query without GROUP BY,
expression contains nonaggregated column
'Student.DepartmentID'

Correct SQL Query

```
SELECT DepartmentID,  
COUNT(StudentID) AS TotalStudents  
FROM Student  
GROUP BY DepartmentID;
```

Output

DepartmentID	TotalStudents
1	2
2	2
3	1

Explanation

- COUNT(StudentID) counts the total number of students.
- GROUP BY DepartmentID groups all students according to their department.
- The query returns one row for each department along with the number of students in that department.
- This query is syntactically correct and produces meaningful grouped results.

3. Correct a faulty SELECT query where column names are misspelled or improperly referenced

ANS:

Introduction

The SELECT statement is used to retrieve data from one or more tables in a database. If a column name is misspelled or incorrectly referenced, SQL generates an error because it cannot identify the specified column. Correct column names and proper table references are essential for successful query execution.

Create Sample Table

```
CREATE TABLE Employee
```

```
(
```

```
    EmpID INT PRIMARY KEY,
```

```
    EmpName VARCHAR(50),
```

```
    Department VARCHAR(50),
```

```
    Salary DECIMAL(10,2)
```

```
);
```

Insert Sample Data

```
INSERT INTO Employee VALUES
```

```
(101,'Rahul','HR',35000),
```

```
(102,'Priya','IT',45000),
```

```
(103,'Arjun','Finance',40000),
```

```
(104,'Sneha','IT',50000);
```

Faulty SELECT Query

```
SELECT EmpNam, Departmnt, Salry
```

```
FROM Employee;
```

Errors Identified

The query contains several mistakes:

- EmpNam is incorrect; the correct column name is EmpName.
- Departmnt is incorrect; the correct column name is Department.

- Salry is incorrect; the correct column name is Salary.

Because these column names do not exist in the table, SQL returns an "Unknown column" error.

Example error:

ERROR 1054 (42S22):

Unknown column 'EmpNam' in 'field list'

Correct SQL Query

```
SELECT EmpName,
       Department,
       Salary
FROM Employee;
```

Output

EmpName	Department	Salary
Rahul	HR	35000.00
Priya	IT	45000.00
Arjun	Finance	40000.00
Sneha	IT	50000.00

Explanation

- The corrected query uses the exact column names defined in the Employee table.
- SQL successfully retrieves the employee name, department, and salary.
- Proper column references prevent syntax errors and ensure accurate results.

4. Fix errors in an SQL query using JOIN, where the join condition is missing or incorrect.

ANS:

Introduction

The **JOIN** clause is used to combine records from two or more tables based on a related column. If the **join condition is missing or incorrect**, SQL may produce incorrect results, duplicate records (Cartesian product), or return no matching records. A proper join condition ensures accurate data retrieval.

Create Tables

Employee Table

```
CREATE TABLE Employee
(
  EmpID INT PRIMARY KEY,
  EmpName VARCHAR(50),
  DeptID INT
);
```

Department Table

```
CREATE TABLE Department
(
  DeptID INT PRIMARY KEY,
  DeptName VARCHAR(50)
);
```

Insert Sample Data

Employee

INSERT INTO Employee VALUES

(101,'Rahul',1),
(102,'Priya',2),
(103,'Arjun',3),
(104,'Sneha',2);

Department

INSERT INTO Department VALUES

(1,'HR'),
(2,'IT'),
(3,'Finance');

Faulty SQL Query

The following query is incorrect because **no JOIN condition is specified**.

```
SELECT EmpName, DeptName  
FROM Employee  
JOIN Department;
```

Error Analysis

The query is incorrect because:

- The **ON** clause is missing.
- SQL does not know how to match rows from the two tables.
- This produces a **Cartesian Product (Cross Join)**, where every employee is matched with every department.
- The result contains incorrect and redundant data.

Correct SQL Query

```
SELECT  
    e.EmpID,  
    e.EmpName,  
    d.DeptName  
FROM Employee e  
JOIN Department d  
ON e.DeptID = d.DeptID;
```

Output

EmpID	EmpName	DeptName
101	Rahul	HR
102	Priya	IT
103	Arjun	Finance
104	Sneha	IT

Explanation

- The **JOIN** combines records from both tables.
- The **ON e.DeptID = d.DeptID** condition correctly matches each employee with their department.
- Table aliases (e and d) make the query shorter and easier to read.
- The corrected query returns only the matching records without duplication.

5. Identify and correct a subquery error where multiple values are returned instead of a single value.

ANS:

Introduction

A subquery is a query written inside another SQL query. When a subquery is used with a single-row operator such as = or <, it must return only one value. If the subquery returns multiple values, SQL generates an error. This issue can be fixed by using the IN operator or modifying the subquery so that it returns only one value.

Create Tables

Department Table

```
CREATE TABLE Department
(
    DeptID INT PRIMARY KEY,
    DeptName VARCHAR(50)
);
```

Employee Table

```
CREATE TABLE Employee
(
    EmpID INT PRIMARY KEY,
    EmpName VARCHAR(50),
    DeptID INT,
    Salary DECIMAL(10,2),
    FOREIGN KEY (DeptID)
    REFERENCES Department(DeptID)
);
```

Insert Sample Data

Department

```
INSERT INTO Department VALUES
(1,'HR'),
(2,'IT'),
(3,'Finance');
```

Employee

```
INSERT INTO Employee VALUES
(101,'Rahul',2,50000),
(102,'Priya',2,55000),
(103,'Arjun',1,45000),
(104,'Sneha',3,60000);
```

Faulty SQL Query

```
SELECT EmpName
FROM Employee
WHERE DeptID =
(
    SELECT DeptID
    FROM Department
);
```


Error Analysis

The subquery:

```
SELECT DeptID FROM Department;
```

returns three values:

1
2
3

However, the operator = expects only one value. Since multiple values are returned, SQL produces an error.

Example error:

ERROR 1242 (21000):

Subquery returns more than 1 row

Correct SQL Query

Use the IN operator instead of =.

```
SELECT EmpName  
FROM Employee  
WHERE DeptID IN  
(  
    SELECT DeptID  
    FROM Department  
);
```

Output

EmpName
Rahul
Priya
Arjun
Sneha

Alternative Solution (If Only One Value Is Required)

If the intention is to retrieve employees from only the IT department, modify the subquery so that it returns a single value.

```
SELECT EmpName  
FROM Employee  
WHERE DeptID =  
(  
    SELECT DeptID  
    FROM Department  
    WHERE DeptName = 'IT'  
);
```

Output

EmpName
Rahul
Priya

Explanation

- The faulty query uses the = operator, which accepts only one value.
- Since the subquery returns multiple department IDs, SQL generates an error.
- Replacing = with IN allows SQL to compare DeptID with multiple values.
- If only one result is expected, modify the subquery using a condition (such as WHERE DeptName = 'IT') so that it returns exactly one row.

6. Debug a VIEW creation statement that contains invalid column references or syntax errors.

ANS:

Introduction

A VIEW is a virtual table created using the CREATE VIEW statement. It stores a SQL query instead of actual data and displays data from one or more tables. If the view contains invalid column names, incorrect table references, or syntax errors, SQL cannot create the view. These errors must be identified and corrected.

Create Sample Table

```
CREATE TABLE Employee
(
    EmpID INT PRIMARY KEY,
    EmpName VARCHAR(50),
    Department VARCHAR(50),
    Salary DECIMAL(10,2)
);
```

Insert Sample Data

```
INSERT INTO Employee VALUES
(101,'Rahul','HR',35000),
(102,'Priya','IT',45000),
(103,'Arjun','Finance',40000),
(104,'Sneha','IT',50000);
```

Faulty VIEW Statement

```
CREATE VIEW Employee_View AS
SELECT EmployeeName, DeptName, Salary
FROM Employee;
```

Errors Identified

The above VIEW statement contains the following errors:

1. EmployeeName is not a valid column name. The correct column is EmpName.
2. DeptName is not a valid column name. The correct column is Department.
3. SQL cannot create the view because the referenced columns do not exist in the Employee table.

Possible Error Message:

ERROR 1054 (42S22):

Unknown column 'EmployeeName' in 'field list'

Correct VIEW Statement

```
CREATE VIEW Employee_View AS
SELECT
    EmpName,
    Department,
    Salary
FROM Employee;
```

Display the View

```
SELECT * FROM Employee_View;
```

Expected Output

EmpName	Department	Salary
Rahul	HR	35000.00
Priya	IT	45000.00
Arjun	Finance	40000.00
Sneha	IT	50000.00

Explanation

- The invalid column name EmployeeName was replaced with the correct column EmpName.
- The invalid column name DeptName was replaced with Department.
- The corrected VIEW statement successfully creates a virtual table that displays employee name, department, and salary.

7. Examine an SQL query with unnecessary conditions and optimize it by simplifying the WHERE clause.

ANS:

Introduction

The WHERE clause is used to filter records in an SQL query. Sometimes, queries contain unnecessary or redundant conditions that make them more complex and may reduce performance. Simplifying the WHERE clause improves readability, reduces execution time, and makes the query easier to maintain.

Create Sample Table

```
CREATE TABLE Employee
```

```
(  
    EmpID INT PRIMARY KEY,  
    EmpName VARCHAR(50),  
    Department VARCHAR(50),  
    Salary DECIMAL(10,2)  
);
```

Insert Sample Data

```
INSERT INTO Employee VALUES  
(101,'Rahul','IT',50000),  
(102,'Priya','HR',45000),  
(103,'Arjun','IT',55000),  
(104,'Sneha','Finance',60000),  
(105,'Kiran','IT',52000);
```

Inefficient SQL Query

The following query contains unnecessary conditions.

```
SELECT *  
FROM Employee  
WHERE Department='IT'  
AND Salary > 40000  
AND Salary > 30000;
```

Analysis of the Query

The query contains redundant conditions:

- Salary > 40000
- Salary > 30000

If a salary is greater than 40000, it is automatically greater than 30000. Therefore, the condition `Salary > 30000` is unnecessary and does not affect the result.

Optimized SQL Query

```
SELECT *  
FROM Employee  
WHERE Department='IT'  
AND Salary > 40000;
```

Expected Output

EmpID	EmpName	Department	Salary
101	Rahul	IT	50000.00
103	Arjun	IT	55000.00
105	Kiran	IT	52000.00

Explanation

- The redundant condition `Salary > 30000` has been removed.
- The optimized query produces the same result as the original query.
- Fewer conditions make the query easier to read and slightly improve performance.

8. Identify violations of integrity constraints in a given table and correct them.

ANS:

Introduction

Integrity constraints are rules that ensure the accuracy, consistency, and reliability of data in a database. Common integrity constraints include PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, and CHECK constraints. Violating these constraints leads to inconsistent or invalid data.

Create Tables

Department Table

```
CREATE TABLE Department  
(  
    DepartmentID INT PRIMARY KEY,  
    DepartmentName VARCHAR(50) NOT NULL  
);
```

Student Table

```
CREATE TABLE Student  
(  
    StudentID INT PRIMARY KEY,  
    StudentName VARCHAR(50) NOT NULL,  
    Email VARCHAR(100) UNIQUE,  
    Age INT CHECK (Age >= 18),  
    DepartmentID INT,  
    FOREIGN KEY (DepartmentID)  
    REFERENCES Department(DepartmentID)  
);
```

Sample Data with Violations

```
INSERT INTO Student VALUES  
(101,'Rahul','rahul@gmail.com',20,1),  
(101,'Priya','rahul@gmail.com',17,5),  
(103,NULL,'arjun@gmail.com',22,2);
```

Identify the Violations

Constraint	Violation	Reason
PRIMARY KEY	StudentID = 101 is repeated	Duplicate primary keys are not allowed.
UNIQUE	Email rahul@gmail.com is repeated	Email addresses must be unique.
CHECK	Age = 17	Age must be 18 or above.
NOT NULL	StudentName is NULL	Student name cannot be empty.
FOREIGN KEY	DepartmentID = 5	Department 5 does not exist in the Department table.

Correct the Data

```
INSERT INTO Student VALUES  
(101,'Rahul','rahul@gmail.com',20,1),  
(102,'Priya','priya@gmail.com',19,2),  
(103,'Arjun','arjun@gmail.com',22,2);
```

Verify the Data

```
SELECT * FROM Student;
```

Expected Output

StudentID	StudentName	Email	Age	DepartmentID
101	Rahul	rahul@gmail.com	20	1
102	Priya	priya@gmail.com	19	2
103	Arjun	arjun@gmail.com	22	2

Explanation of Corrections

- PRIMARY KEY: Changed the duplicate StudentID from 101 to 102.
- UNIQUE: Replaced the duplicate email with priya@gmail.com.
- CHECK: Updated the age from 17 to 19.
- NOT NULL: Replaced the NULL value in StudentName with Arjun.
- FOREIGN KEY: Changed DepartmentID from 5 to 2, which exists in the Department table.

9. Fix a relation that violates 1NF or 2NF by removing repeating groups or partial dependencies

ANS:

Introduction

Normalization is the process of organizing data to reduce redundancy and improve data integrity.

- First Normal Form (1NF) removes repeating groups and ensures that each column contains atomic (single) values.
- Second Normal Form (2NF) removes partial dependencies by ensuring that all non-key attributes depend on the entire primary key.

Unnormalized Relation (UNF)

Consider the following Student table:

StudentID	StudentName	Courses
S101	Rahul	DBMS, Python
S102	Priya	Java
S103	Arjun	DBMS, Java

Here, the Courses column contains multiple values, which violates 1NF.

Problems in the Relation

The table has the following issues:

- Repeating groups (multiple courses in one column).
- Data redundancy.
- Difficult to search and update data.
- Insertion and deletion anomalies.

Convert to First Normal Form (1NF)

Split the repeating values so that each row contains only one course.

StudentID	StudentName	Course
S101	Rahul	DBMS
S101	Rahul	Python
S102	Priya	Java
S103	Arjun	DBMS
S103	Arjun	Java

Now every field contains only one value, so the relation satisfies 1NF.

Relation in 2NF

Assume the primary key is (StudentID, Course).

Some attributes depend only on StudentID:

- $\text{StudentID} \rightarrow \text{StudentName}$

This is a partial dependency, so the relation violates 2NF.

Decompose into 2NF

Student Table

StudentID (PK)	StudentName
S101	Rahul
S102	Priya
S103	Arjun

Enrollment Table

StudentID (FK)	Course
S101	DBMS
S101	Python
S102	Java
S103	DBMS
S103	Java

Final 2NF Schema

Student(StudentID, StudentName)

Enrollment(StudentID, Course)

Explanation

- 1NF removes repeating groups by storing one course per row.
- 2NF removes partial dependencies by separating student details from course enrollment.
- Each non-key attribute now depends on the whole primary key.

10. Identify redundant attributes in a relation and apply basic normalization (up to 3NF) to improve design.

ANS:

Introduction

Normalization is the process of organizing data to reduce redundancy and improve data integrity. Redundant attributes cause repeated information, which leads to update, insertion, and deletion anomalies. By applying **First Normal Form (1NF)**, **Second Normal Form (2NF)**, and **Third Normal Form (3NF)**, the database becomes more efficient and consistent.

Given Relation

Student_Course

StudentID	StudentName	Department	CourseID	CourseName	FacultyID	FacultyName
S101	Rahul	CSE	C101	DBMS	F101	Dr. Kumar
S101	Rahul	CSE	C102	Python	F102	Dr. Meena
S102	Priya	ECE	C101	DBMS	F101	Dr. Kumar

Identify Redundant Attributes

The following attributes are repeated unnecessarily:

- **StudentName** is repeated for the same StudentID.
- **Department** is repeated for the same StudentID.
- **CourseName** is repeated for the same CourseID.
- **FacultyName** is repeated for the same FacultyID.

These redundancies increase storage and lead to anomalies.

Problems in the Relation

- **Data Redundancy:** Student, course, and faculty details are stored multiple times.
- **Update Anomaly:** Changing a faculty or student name requires updating several rows.
- **Insertion Anomaly:** A new course cannot be added unless a student is enrolled.
- **Deletion Anomaly:** Deleting the last enrollment of a course may remove course information.

First Normal Form (1NF)

Each attribute contains a single (atomic) value.

StudentID	StudentName	Department	CourseID	CourseName	FacultyID	FacultyName
S101	Rahul	CSE	C101	DBMS	F101	Dr. Kumar
S101	Rahul	CSE	C102	Python	F102	Dr. Meena
S102	Priya	ECE	C101	DBMS	F101	Dr. Kumar

Primary Key: (StudentID, CourseID)

Second Normal Form (2NF)

Remove partial dependencies.

Student Table

StudentID (PK)	StudentName	Department
S101	Rahul	CSE
S102	Priya	ECE

Course Table

CourseID (PK)	CourseName	FacultyID	FacultyName
C101	DBMS	F101	Dr. Kumar
C102	Python	F102	Dr. Meena

Enrollment Table

StudentID (FK)	CourseID (FK)
S101	C101
S101	C102
S102	C101

Third Normal Form (3NF)

Remove transitive dependency by separating faculty details.

Student

StudentID	StudentName	Department
S101	Rahul	CSE
S102	Priya	ECE

Faculty

FacultyID	FacultyName
F101	Dr. Kumar
F102	Dr. Meena

Course

CourseID	CourseName	FacultyID
C101	DBMS	F101
C102	Python	F102

Enrollment

StudentID	CourseID
S101	C101
S101	C102
S102	C101

Final 3NF Schema

Student(StudentID, StudentName, Department)

Faculty(FacultyID, FacultyName)

Course(CourseID, CourseName, FacultyID)

Enrollment(StudentID, CourseID)

Explanation

- **1NF:** Ensures all attributes contain atomic values.
- **2NF:** Removes partial dependencies by separating student and course information.
- **3NF:** Removes transitive dependencies by storing faculty information in a separate table.
- Each table now stores only data related to its primary key.